— ARS · MAGNA —

OPUS

A Bio-Inspired Multi-Agent Swarm Architecture for Collective Reasoning

Technical Whitepaper · v0.1
Magnum Opus · MMXXVI

Abstract

Opus is a multi-agent reasoning architecture inspired by the collective behaviour of biological swarms — most directly, the honeybee colony. Where conventional language-model deployments rely on a single model producing a single answer through a single chain of thought, Opus distributes reasoning across a population of specialised agents that operate in parallel, communicate indirectly through a shared cognitive substrate, and converge on a final output through a structured consensus mechanism.

This document describes the technical architecture of Opus, the rationale for its design, and the token-based economic model that governs early access and ongoing usage. Opus is presently in active development as a live-coded AI art project, broadcast publicly. The system is not a model — it is a colony. What follows is the schematic of that colony.

— ✦ —

1. Introduction

1.1 The Problem with Solitary Reasoning

A modern large language model, however capable, reasons as a monologue. It produces a single trajectory of thought, biased by the first tokens it samples and constrained by the assumptions implicit in its prompt. When that trajectory is wrong, the entire output is wrong. There is no internal dissent, no parallel hypothesis, no mechanism by which the model's first instinct can be challenged before it hardens into an answer.

This is not a limitation of any specific model. It is a structural property of single-agent inference. The model has one voice, and the voice does not argue with itself.

Where one model answers, the swarm deliberates. Where one model guesses, the swarm proves.

1.2 The Hypothesis

Opus proposes that reasoning is better modelled as a dialectic among many minds than as the output of one. A swarm of specialised agents — each pursuing a different hypothesis, each subject to critique by its peers, each contributing only what survives evaluation — produces a different kind of output altogether. Not a guess refined into prose, but a consensus refined out of disagreement.

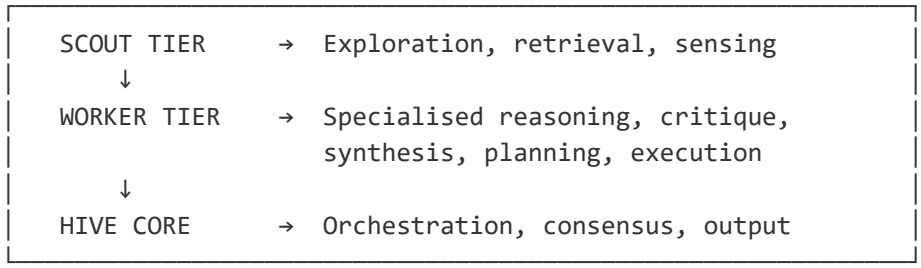
This is the same principle by which honeybees select a new nest site, by which ant colonies route around obstacles, and by which scientific communities converge on truth. It is also the principle behind classical multi-agent systems research, distributed consensus algorithms, and ensemble methods in machine learning. Opus synthesises these lineages into a single working system, built on top of frontier language models.

— ✦ —

2. System Architecture

Opus is structured as a hexagonal lattice of agents organised into three concentric tiers, coordinated by a central Hive Core. The lattice is more than ornamental: it provides natural topological neighbourhoods for stigmergic communication and visualises the system's biological inspiration. The three tiers are Scout, Worker, and Core.

2.1 Topology at a Glance



2.2 Scout Agents

Scouts are lightweight, high-throughput agents tasked with gathering raw material. They retrieve documents from the vector and graph stores, query external APIs, run web searches, sample the local filesystem, and otherwise furnish the swarm with context. Scouts do not reason about the problem; they enlarge the perimeter of available information. A typical Opus deployment runs between eight and thirty-two Scouts in parallel, each pursuing an orthogonal angle of inquiry.

2.3 Worker Agents

Workers are the reasoning core of the swarm. Each Worker is a specialised role — Researcher, Critic, Synthesiser, Planner, Executor, Verifier — and each operates with its own prompt, temperature, and model assignment. Researchers generate hypotheses; Critics attack them; Synthesisers merge surviving fragments; Planners structure the final solution; Executors carry out concrete actions; Verifiers re-examine the result against the original problem.

Workers do not communicate directly. They write to and read from the shared cognitive substrate (see §3), and their interactions are mediated entirely through that substrate. This is the stigmergic principle borrowed from social insects: agents leave traces in a shared environment, and other agents respond to those traces.

2.4 The Hive Core

The Hive Core is the orchestration and consensus layer. It does not itself reason about the problem; instead, it manages the lifecycle of the swarm — spawning agents, allocating compute, enforcing deadlines, pruning underperforming branches, and ultimately collapsing the swarm's many partial outputs into a single response. The Core is also where the consensus mechanism (see §4) is executed.

— ✦ —

3. The Cognitive Substrate

The defining engineering challenge of any multi-agent system is coordination. Direct agent-to-agent messaging produces $O(n^2)$ communication complexity and tightly couples agents to one another's internal state. Opus avoids this by adopting the blackboard pattern, augmented with vector and graph stores, and grounded in the biological metaphor of stigmergy: communication through modification of a shared environment.

3.1 The Blackboard

At the centre of every Opus run is a structured key-value store known as the Blackboard. Each Worker writes its hypotheses, critiques, and partial syntheses to the Blackboard as typed records, tagged with provenance, confidence, and timestamp. Other Workers read the Blackboard at every step and condition their reasoning on its current state. The Blackboard is backed by a CRDT (Conflict-free Replicated Data Type) so that concurrent writes from many agents never produce inconsistent states.

3.2 Vector Memory

Long-form context — documents retrieved by Scouts, prior transcripts, codebases, reference material — is embedded and stored in a vector index (Qdrant in the reference implementation, with optional Weaviate or pgvector backends). Workers query this index by semantic similarity rather than keyword, allowing them to pull in only the context relevant to their current sub-problem.

3.3 Graph Memory

Relational structure — dependencies, citations, causal chains, code graphs, ontologies — is stored in a graph database (Neo4j). Where vector memory captures "what is similar," graph memory captures "what is connected." Many of the failure modes of single-model reasoning trace to a model conflating these two questions; Opus separates them at the storage layer.

3.4 Provenance and Trace

Every record on the Blackboard carries a provenance trail: which agent produced it, from which prior records, using which model and at what cost. The trace is fully reconstructible after the fact, which makes the system auditable, debuggable, and — critically — pricable. The token economics described in §6 depend on this provenance layer.

— ✦ —

4. Consensus and Convergence

A swarm that produces a hundred candidate answers is not useful unless it can collapse them into one. Opus's consensus layer is the mechanism by which disagreement among Workers becomes a single, defensible output.

4.1 Weighted Borda Aggregation

Each Worker that produces a candidate answer also ranks the candidates it can see from its peers. These rankings are aggregated using a weighted Borda count, where weights reflect the historical reliability of each Worker on similar problems. The result is a preference ordering over candidates that does not collapse to mere majority vote and is robust to clusters of correlated agents.

4.2 LLM-as-Judge Tie-Breaking

Where Borda aggregation produces a near-tie, a separate Judge agent is invoked. The Judge sees the top-ranked candidates and the trace that produced each, but does not see Worker identities. It selects the candidate it can most rigorously defend against the strongest critique from the trace. This step is deliberately expensive and is invoked only when necessary.

4.3 Verifier Pass

The selected candidate is finally subjected to a Verifier pass: an independent agent that re-reads the original problem and the proposed answer, and attempts to falsify the answer. If falsification succeeds, the swarm re-enters deliberation with the falsification as a new constraint. If falsification fails after a configurable number of attempts, the answer is finalised.

— ✦ —

5. Technical Stack

Opus is implemented in Python with a TypeScript front-end. The reference deployment uses the following components:

LLM Core	Claude Opus 4.7 (Workers, Judge, Verifier) and Claude Sonnet 4.6 (Scouts, lightweight critique). Cost-tiered specialisation per role.
Orchestration	Custom event-driven message bus on top of asyncio and anyio; agent lifecycle managed by a supervisor pattern.
Vector Store	Qdrant (default), with adapters for Weaviate and pgvector.
Graph Store	Neo4j Community Edition for relational and causal memory.
Blackboard	Redis-backed CRDT store (Yjs / Automerge interoperable) for concurrent writes.
Distributed Runtime	Modal for ephemeral agent execution; Ray for long-running cluster deployments.
Observability	OpenTelemetry traces, per-agent reasoning logs, token-level cost provenance, live dashboard.
Front-end	Next.js 14 (App Router) with React Three Fiber for the live armillary visualisation.
Deployment	Vercel (front-end edge), Fly.io (agent runtime), AWS S3 (artefact storage).

— ✦ —

6. Token Economics

Opus is computationally expensive by design. A single query may invoke dozens of Worker agents, each consuming inference tokens; multiple Scouts running web search and retrieval; and a Judge plus Verifier pass on top. The cost of producing a high-quality consensus answer is several orders of magnitude greater than the cost of a single-model query — and that cost must be reflected honestly in the system's economics.

Rather than bury this cost behind a flat subscription, Opus uses a transparent token-based model. Every action the swarm takes is metered, priced, and surfaced to the user.

6.1 The OPUS Token

OPUS is the native unit of account for the system. One OPUS token corresponds to a fixed quantum of swarm work — defined as the inference, storage, and orchestration cost of a single Worker-step at the reference model tier. All system costs, regardless of underlying provider, are translated into OPUS at run-time using a published conversion table.

6.2 Early Access

Access to Opus during the closed beta phase is gated by token holdings. To enter the beta, a participant must hold a minimum allocation of OPUS tokens, which serves both as a commitment device (filtering for users who will engage seriously with the system) and as the working balance from which their swarm runs are drawn.

Early-access allocations are distributed in cohorts. Each cohort receives a fixed number of tokens at a published rate, with later cohorts paying more per token. This is not speculative pricing — it reflects the increasing marginal cost of supporting additional users on a system whose compute requirements scale super-linearly with population.

Cohort	Phase	Allocation	Rate (per OPUS)
Genesis	Closed alpha	Invitation only	Reserved
Cohort I	Private beta	100,000 OPUS	Tier 1
Cohort II	Public beta	500,000 OPUS	Tier 2
Cohort III	Open access	Floating supply	Market rate

6.3 Continuing Use: Tokens Burn

Continued use of Opus consumes tokens. This is not a metaphor: every swarm run debits the user's OPUS balance in proportion to the work performed. Tokens are not refunded, and tokens are not spent elsewhere — they are burned, removed permanently from circulation.

The burn model has three consequences. First, it ties cost directly to value: a user who runs a hundred trivial queries pays for a hundred trivial queries, while a user who runs one deep research swarm pays for the depth they actually invoked. Second, it produces a deflationary pressure on the token supply, which compensates early-access participants for the risk of supporting an unproven system. Third, it provides an honest accounting of compute: the system cannot pretend to be cheap when it is not, and the user can see exactly what their reasoning cost.

Cost components per run

- Worker-steps: one OPUS per Worker invocation at the reference tier; scaled by model used.
- Scout queries: a fractional OPUS per retrieval, depending on backend (vector, graph, web).
- Judge invocations: priced at four times the Worker rate, reflecting the deeper context window.
- Verifier passes: priced equivalently to a Worker, applied per attempt.
- Storage of long-lived artefacts: a small recurring cost per gigabyte-month, debited daily.

Worked example

A standard research query of moderate complexity might involve sixteen Scouts, twenty-four Workers across three rounds of deliberation, one Judge invocation, and two Verifier passes. At the reference rate, this resolves to approximately 28 OPUS — visible to the user before the swarm is dispatched, and confirmable afterwards in the run trace.

6.4 Transparency

Every charge against an OPUS balance is itemised in the run trace. Users see, line by line, which agents were invoked, which models they used, how many tokens of inference they consumed, and how that translated into the final OPUS cost. There are no opaque platform fees, no hidden routing costs, and no rounding in the platform's favour. The provenance system described in §3.4 is the foundation of this transparency.

— ✦ —

7. Roadmap

Opus is being built in public, live-coded daily. The roadmap below reflects current intent and is subject to revision based on what the build reveals.

- Phase α — Genesis (current). Single-machine swarm, three Worker roles, naive Borda consensus, manual orchestration. Live build broadcast daily.
- Phase β — Cohort I. Distributed runtime on Modal, six specialised Worker roles, full stigmergic Blackboard, OPUS token issuance to first cohort.
- Phase γ — Cohort II. Public beta. Open enrolment to Cohort II, full Judge/Verifier pipeline, observability dashboard, public run gallery.
- Phase δ — Open Access. Floating token supply, third-party agent integrations, marketplace for community-built Worker roles, full whitepaper v1.0.

— ✦ —

8. Lineage

Opus does not invent. It synthesises a long lineage of human attempts to build collective intelligence.

The architecture descends, at the conceptual level, from the combinatorial wheels of Ramon Llull's thirteenth-century Ars Magna — itself the first serious attempt to mechanise reasoning by combining many concepts into one apparatus. It descends, in algorithmic form, from Marco Dorigo's ant colony optimisation, Eberhart and Kennedy's particle swarm work, and Reynolds's boids. It descends, in software-engineering form, from the blackboard architectures of Hearsay-II and from contemporary multi-agent

LLM frameworks. And it descends, in spirit, from the alchemical magnum opus: the slow refinement of base material into something rare.

Solve et coagula. Dissolve and re-form. The colony reasons by separation and recombination.

— ✦ —

9. Closing

This whitepaper describes a system that is, at the time of writing, partially built and openly broadcast. It is not a product announcement. It is the schematic of an ongoing experiment — an attempt to ask whether a swarm of cooperating, disagreeing, refining minds can produce something a single mind cannot.

Opus is the work, and the work is in progress.

— ARS · MAGNA —